

面向 LLM / Agent Systems 科研的深度学习零基础课

## 第 08 讲

### 实验设计：多层 Linear 为什么仍可折叠

实验目标：用控制实验验证两层/五层 affine 的精确折叠，并用 ReLU 对照观察非线性为何破坏这种折叠。

实验设计文档

本课程采用 Just-in-Time Math：数学只讲到足以理解深度学习，并通过手算 + NumPy + 后续 PyTorch 建立底层直觉。

## 1 实验背景

理论上：

$$H = XW_1 + b_1$$

$$Y = HW_2 + b_2$$

可以写成：

$$Y = X(W_1W_2) + (b_1W_2 + b_2)$$

这意味着：没有 Activation 或其他非线性操作时，堆叠多个 Linear/Affine 并不会自动产生新的非线性函数类别。

## 2 研究问题

1. 两层一维 affine 是否能精确合并？
2. 两层矩阵 affine 是否对任意 Batch 输入得到完全相同输出？
3. 五层 affine 是否仍可逐层折叠？
4. 加入 ReLU 后，一条全局 affine 是否还能精确替代？
5. 一个简单  $y = x^2$  目标为何无法由单一 affine 表达？
6. PyTorch 两个 `nn.Linear` 无 Activation 时是否也能精确折叠？

## 3 实验假设

- H1：两层标量 affine 与折叠后的单层完全一致；
- H2：矩阵两层输出与  $W_{eq}, b_{eq}$  输出数值一致；
- H3：五层 affine 折叠误差仅为浮点舍入级别；
- H4：加入 ReLU 后，最佳单一直线仍会有显著残差；
- H5：PyTorch 与 NumPy 的结论一致。

## 4 实验环境

- Python 3.x；
- NumPy；
- Matplotlib；
- PyTorch 可选，用于最后的框架对照。

运行：

```
python lesson08_why_nonlinearity_experiment.py
```

## 5 实验 1：标量两层折叠

设置：

$$h = 2x + 1$$

$$y = 3h - 4$$

理论折叠：

$$y = 6x - 1$$

输入多个  $x$ ：

$$[-2, -1, 0, 1, 2]$$

比较两种算法输出。

通过标准：np.allclose=True。

## 6 实验 2：矩阵两层折叠

输入：

$$X : (3, 2)$$

第一层：

$$W_1 : (2, 3), \quad b_1 : (3,)$$

第二层：

$$W_2 : (3, 2), \quad b_2 : (2,)$$

计算两条路径：

$$Y_{two} = (XW_1 + b_1)W_2 + b_2$$

和：

$$Y_{one} = X(W_1W_2) + (b_1W_2 + b_2)$$

实际运行结果：完全一致。

## 7 实验 3：五层 affine 折叠

随机生成维度：

$$3 \rightarrow 5 \rightarrow 4 \rightarrow 6 \rightarrow 2 \rightarrow 3$$

让 7 个样本通过五层网络，再递归计算等效  $W, b$ 。

实际结果：

$$\max |Y_{deep} - Y_{fold}| \approx 3.55 \times 10^{-15}$$

属于浮点精度误差。

### 核心概念

这不是“近似上差不多”，而是数学上应当完全相等；计算机里出现极小误差是浮点数表示造成的。

## 8 实验 4：加入 ReLU 的对照

设置：

$$y = 3 \operatorname{ReLU}(2x + 1) - 4$$

在  $[-3, 3]$  上采样 301 个点。

再用最小二乘找到最佳 affine：

$$y = ax + b$$

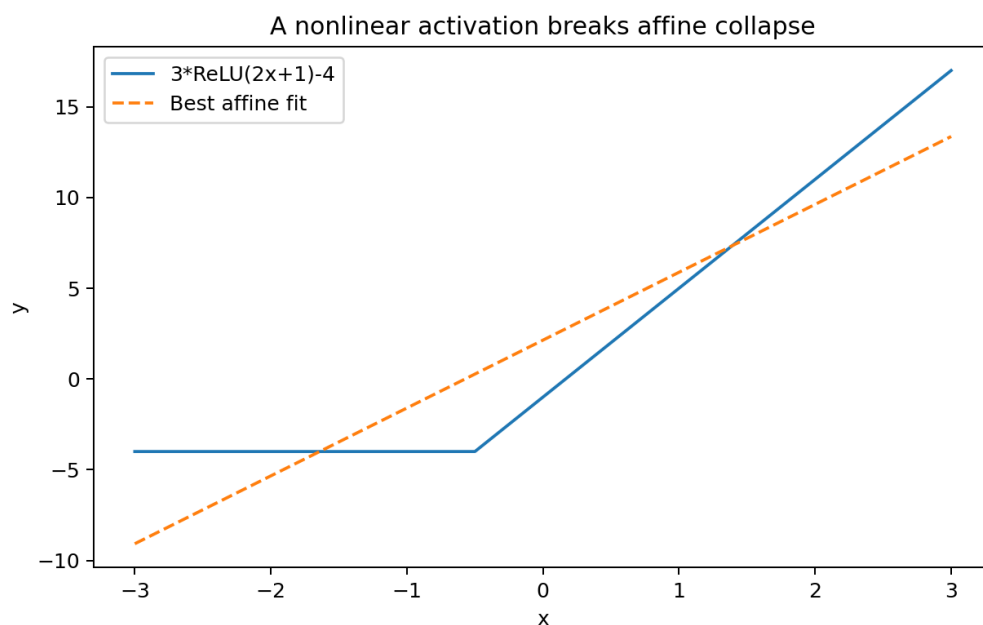
实际得到：

$$a \approx 3.7406, \quad b \approx 2.1395$$

$$MSE \approx 6.2481$$

最大绝对误差约：

$$5.0824$$



**科研视角**

这个实验最重要的不是 MSE 数值，而是“结构性误差”：即使我们找到最佳直线，也无法把有折点的函数全局精确表示。这是函数类别不匹配，而不是简单的参数没有调好。

## 9 实验 5：三个点证明 $x^2$ 不能被一条直线表达

取：

$$(-1, 1), (0, 0), (1, 1)$$

通过  $(0, 0)$  与  $(1, 1)$  的直线只能是  $y = x$ ，但在  $x = -1$  会输出  $-1$ ，与目标  $1$  冲突。

这是一个无需训练算法的“反例证明”。

## 10 实验 6：PyTorch 对照

建立：

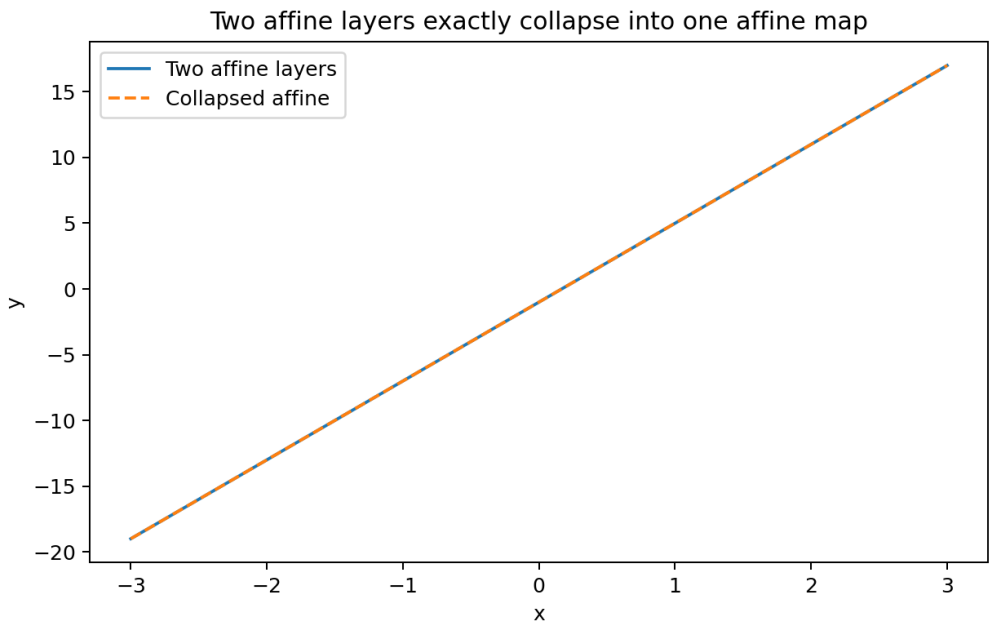
```
l1 = nn.Linear(2, 3)
l2 = nn.Linear(3, 2)
y = l2(l1(x))
```

PyTorch 的 Weight 存储为  $(out, in)$ ，折叠公式写成：

```
W_eq = l2.weight @ l1.weight
b_eq = l1.bias @ l2.weight.T + l2.bias
```

实际运行：两层与折叠结果一致，**True**。

11 实验 7：可视化两层与折叠层



图中两条曲线重合，虚线几乎看不出来，因为它们数值完全相同。

12 控制变量与实验设计说明

| 实验               | 改变的因素    | 保持不变的因素   |
|------------------|----------|-----------|
| 两层 vs 折叠         | 参数表达形式   | 输入、最终函数   |
| 五层 vs 折叠         | 网络深度表达形式 | 等效函数      |
| 无 ReLU vs 有 ReLU | 是否加入非线性  | 外层系数与输入范围 |
| PyTorch 对照       | 实现框架     | 数学函数      |

13 实际结果摘要

- 标量两层 vs 单层：True；
- 矩阵两层 vs 单层：True；
- 五层折叠最大绝对误差约  $3.55 \times 10^{-15}$ ；
- 带 ReLU 的函数无法被最佳 affine 精确替代；
- $x^2$  三点反例说明 affine 表达限制；
- PyTorch 两层 Linear 折叠：True。

## 14 实验记录模板

| 实验         | 运行前预测 | 实际结果 | 解释 |
|------------|-------|------|----|
| 标量两层折叠     |       |      |    |
| 矩阵两层折叠     |       |      |    |
| 五层折叠       |       |      |    |
| ReLU 对照    |       |      |    |
| $x^2$ 三点反例 |       |      |    |
| PyTorch 对照 |       |      |    |

## 15 实验结论

### 核心概念

实验支持理论结论：多个纯 Linear/Affine 层之间如果没有非线性操作，固定参数后的输入-输出函数仍可折叠为一个 affine；加入 ReLU 后出现折点，简单的全局 affine 折叠不再成立。

## 16 科研方法迁移

这一讲训练三个重要习惯：

1. 先做代数推导，再做数值验证；
2. 用极小反例判断模型表达能力边界；
3. 区分“函数类别”“参数化结构”“训练动力学”三个不同层面。